Determinants of Hyperparameter Invariance in Connectome Reservoir Computing

Working Draft (Feb 2026)

A Thesis
Presented to
The Division of Mathematical and Natural Sciences
Reed College

In Partial Fulfillment
of the Requirements for the Degree
Bachelor of Arts

Miles Walter Churchland

May 2026

Approved for the Division
(Mathematics)

Greg Anderson

Draft status

This is an early working draft circulated for context and feedback. Many sections are incomplete; wording, figures, and statistical analyses may change. Please treat claims in the Results as preliminary pending final checks and additional controls.

Acknowledgements

Firstly, I want to thank my parents Anne Churchland and Michael Brodesky. Their endless support, advice, and work mentality have enabled the creation of this work. Secondly, I would like to thank my grandmother, Patricia Churchland

List of Abbreviations

ANN	Artificial Neural Network
C. elegans	*Caenorhabditis elegans*
D. melanogaster	*Drosophila melanogaster*
Dale's law	Neurons are exclusively excitatory or inhibitory
E/I	Excitatory/Inhibitory
EI ratio	Excitatory-to-inhibitory neuron ratio
ESN	Echo State Network
GR	Generalisation Rank (noise robustness)
Gym	Gymnasium RL environment toolkit
HPC	High-Performance Computing
IPC	Information Processing Capacity
KR	Kernel Rank (Kernel Quality; nonlinear separability)
LSTM	Long Short-Term Memory
MC	Memory Capacity (linear/nonlinear)
MG	Mackey–Glass time series
NARMA-N	Nonlinear AutoRegressive Moving Average (order N)
NRMSE	Normalised Root Mean Squared Error
PPO	Proximal Policy Optimisation
RC	Reservoir Computing
RNN	Recurrent Neural Network
SNN	Spiking Neural Network
SR	Spectral radius ($\rho(W)$)
SW	Small-World (graph property)
WS	Watts–Strogatz small-world model
W	Reservoir (recurrent) weight matrix
W^{in}	Input weight matrix
CAN	canal-associated neurons; linked to the excretory canal (CANL/CANR).
L/R	Suffixes used throughout: Left/Right

Contents

1 Introduction **1**
1.1 Neuroscience Terms Explained for Computer Scientists 1

2 Echo State Networks **3**
2.1 Echo State Networks: an Introduction 3
2.2 Hyper-Parameters 4

3 Background **7**
3.1 The C. elegans Connectome 7
3.1.1 Connectome 7
3.1.2 Omission of CANL/CANR 8
3.1.3 Isolated nodes 8
3.1.4 Collective dynamics of small-world networks 8
3.1.5 A small-world topology enhances the echo state property and signal propagation in reservoir computing 9
3.2 Task Agnostic Network Statistics 9
3.2.1 Motivation 9
3.2.2 Kernel Rank 10
3.2.3 Generalization Rank 10
3.2.4 Memory Capacity 11
3.2.5 Information Processing Capacity 12
3.3 Invariance across Hyper-parameters 14
3.3.1 Hyperparameter Sweeps 14

4 Methods **17**
4.1 Network Architectures 17
4.1.1 Celegan connectome 17
4.1.2 Connection-Shuffle 17
4.1.3 Weight-Shuffle 17
4.1.4 Connection-shuffle with Weight-Shuffle 18
4.1.5 C. elegans connections with Gaussian Weights 18
4.1.6 Celegan matched Erdos Renyl Model With Gaussian Weights 18
4.1.7 Celegan matched Watts–Strogatz Model With Gaussian Weights 19
4.1.8 Celegan architecture with Gaussian weights using original Connection Signs 19

4.1.9 Celegan architecture with uniform weights using original Connection Signs . . . 19
4.1.10 Celegan architecture with sampled weights using original Connection Signs . . . 19
4.1.11 Celegan architecture with +1/-1 weights using original Connection Signs . . . 20
4.1.12 Celegan architecture with +1/-1 weights using original Connection Sign balance with the signs shuffled . . . 20
4.2 Trial definition . . . 20
4.3 Equivalence testing . . . 21

5 Results . . . 23
5.1 Weight magnitude distribution causally Both modulates performance and hyperparameter sensitivity . . . 24
5.2 Global neurotransmitter balance, not synapse-specific sign placement, determines invariance . . . 24
5.3 Global neurotransmitter balance is a causal determinant of invariance 26

6 AI Usage Disclosure . . . 31

List of Tables

5.1 Pairwise equivalence tests (TOST) across modes within each task agnostic performance metric. 30

List of Figures

2.1 A basic echo state network architecture. W^{out} edges are trainable. W^{in} and W^{res} are fixed after the network architecture is created and are not updated during training. 4

5.1 Mean coefficient of variation (z) and mean weight magnitude (y) as a function of perturbation magnitude Λ (x-axis, log scale). Nonzero binary weights are perturbed according to $w'_{ij} = \text{sign}(w_{ij})(1+\Lambda|\mathcal{N}(0,1)|)$, preserving connectivity and sign structure. Increasing Λ increases both the mean weight magnitude and the coefficient of variation across hyperparameter settings. 25

5.2 Mean computational performance (z) and mean weight magnitude (y) as a function of perturbation magnitude Λ (x-axis, log scale), with connectivity and sign structure fixed. Increasing weight magnitude distribution scale increases achievable performance, followed by saturation consistent with spectral radius normalization. 25

5.3 Mean computational metrics as a function of the fraction of synapse signs flipped from the original connectome using binary weights. Increasing sign randomization shifts the network from a low-variance, invariant regime toward a higher-performance but less invariant regime. $N = 250$ for each sign fraction 27

5.4 Mean computational performance and invariance metrics as a function of the fraction of flipped synapse signs. The original biological sign structure ($f = 0$) exhibits maximal invariance, while increasing sign randomization improves performance but increases variability. $N = 250$ for each sign fraction 28

5.5 Invariance dispersion across architectures 29

Abstract

Biological neural circuits remain in functional computational ranges despite substantial variability in synaptic strengths, neuron leak, and inputs. In reservoir computing, performance and stability depend sensitively on hyperparameters such as spectral radius, input scaling, and leak rate; however, it is unclear which network properties govern invariance to these parameters. In this work, we investigate the determinants of hyperparameter invariance using the *C. elegans* connectome as a biologically grounded reservoir model.

We quantify computational capability using task-agnostic metrics—Memory Capacity (MC), Information Processing Capacity (IPC), Kernel Rank (KR), and Generalization Rank (GR)—and measure invariance as the coefficient of variation of these metrics under hyperparameter sweeps. We construct controlled perturbations that independently alter connectivity topology, synaptic weight magnitude distributions, and excitatory/inhibitory sign structure while preserving other structural properties. This allows isolation of the causal contributions of topology, magnitude statistics, and global sign balance to computational invariance.

Preliminary results suggest that global sign balance and weight magnitude statistics account for a large fraction of the observed invariance.

Dedication

You can have a dedication here if you wish.

Chapter 1

Introduction

Picture a pool of water, with no waves on it–this is a compute reservoir; our computational substrate. Now imagine dropping a stone into the pool–our input–observing ensuing waves–our output. One could use the pool to perform computational tasks; dropping multiple stones in the pool in order to compute. Alternatively one could measure the computational power of the pool without regard to a specific task, this is what this work explores.

1.1 Neuroscience Terms Explained for Computer Scientists

- **neuron** – basic building block of neural circuits. In this work, neurons will be represented with a node on a graph that has both activation function and internal state (representing membrane potential).
- **synapse** – Another basic building block of neural circuits. They will be represented by a directed edge with a weight connecting two neurons.
- **excitatory neuron** – a neuron with only positive weight outgoing edges
- **inhibitory neuron** – a neuron with only negative weight outgoing edges
- **E/I balance** – the ratio of excitatory neurons to inhibitory neurons
- **neurotransmitter** – chemicals that allow neurons to communicate with each other–transmitted between synapsesSheffler et al., 2025
- **gap junction** – a type of synapse that connects the cytoplasm of two adjacent cells; bidirectional with the same weight for both sides. In our modeling these are either removed or treated as a two distinct positive signed edges
- **neuromuscular junction** – a specialized synapse where a motor neuron communicates with a muscle fiber. These directed edges are directed to non-neural cells and hence are not included in our graph

- **connectome** – a wiring diagram of a biological neural network. This typically contains all neurons and synapses along with edge weights
- **sensory neuron** – a neuron that receives sensory input. During typical biological use these can be thought of as "inputs" to the network.
- **motor neuron** – a neuron that passes signal to the motor cells. During typical biological use these can be thought of as the "outputs" of the network
- **synaptic terminals** – bulbous endings of a neuron's axon where neurotransmitters are transferred to other cells (neurons, muscles), we use the number of these as a proxy for edge weight.
- **membrane potential** – The membrane potential is the difference in electrical charge between the inside and outside of a neuron, forming the basis for neural activity.Henley, 2021 For our purposes, this is the internal state of the neuron representing how much our activation function will be saturated
- **small-worldness** – a property commonly observed in biological networks. A graph that exhibits the small world property has both high clustering coefficient and short average path length. It is claimed that it makes information spread more efficient, and makes the network more robustBassett and Bullmore, 2007; Latora and Marchiori, 2001.

Chapter 2

Echo State Networks

Let $u_t \in \mathbb{R}^d$ be input, $x_t \in \mathbb{R}^N$ the reservoir state, and $y_t \in \mathbb{R}^p$ the output. With leak $\alpha \in (0, 1]$:

$$\tilde{x}_{t+1} = \tanh(W_{\text{res}} x_t + W_{\text{in}} u_{t+1} + b + \xi_t)\,, \tag{2.1}$$

$$x_{t+1} = (1 - \alpha)\, x_t + \alpha\, \tilde{x}_{t+1}, \tag{2.2}$$

$$y_t = W_{\text{out}}\, [x_t; u_t; 1], \tag{2.3}$$

where tanh is elementwise, b is not included in our model ,ξ_t is optional i.i.d. noise, $W_{\text{res}} \in \mathbb{R}^{N \times N}$ is in our case sparse, and $W_{\text{out}} \in \mathbb{R}^{p \times (N+d+1)}$ is trained.

2.1 Echo State Networks: an Introduction

Echo state networks are discrete time recurrent reservoir computers, introduced by Jaeger Jaeger, 2001. "RC bootcamp", 2026 They have memory, due the recurrence, and are particularly useful in time series analysisLukoševičius et al., 2012 and chaotic system prediction problemsJaeger and Haas, 2004. For readers unfamiliar with reservoir computing, we recommend the tutorial-style introduction by Inoue et al. Inoue et al., 2026 and the accompanying interactive RC Bootcamp notebooks. They have K input units, N internal network (more colloquially known as reservoir network or reservoir) units and L output units. We have 3 sets of weights, we have input weightsW_{ij}^{in} with size $K \times N$ we have the internal weights (W_{ij}^{res}) with size $N \times N$ and we have the output weights, W_{ij}^{out}, with size $N \times L$. Jaeger, 2001. The reservoir acts as a nonlinear expansion and as a memory of the input Lukoševičius, 2012. When training these networks we train solely W_{ij}^{out}, as opposed to training all weights like is typically done in artificial neural networks. When training these networks we "tap" in to the dynamics of the "reservoir" by linearly combining different neuron states, hence we exclusively train W_{ij}^{out}, as opposed to training all weights like is typically done in standard neural networks.

For this approach to work, our internal network must satisfy the echo-state property. This property is that asymptotically the state of a reservoir $x(n)$ is uniquely determined by the input history u_t not the initial state of the network (x_0). A good way to explain this intuitively a house with a thermostat, imagine two identical houses

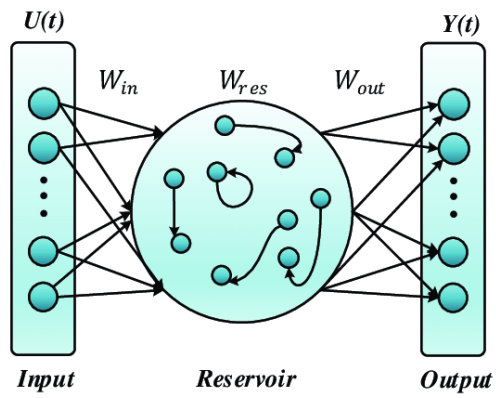

Figure 2.1: A basic echo state network architecture. W^{out} edges are trainable. W^{in} and W^{res} are fixed after the network architecture is created and are not updated during training.

with the same target temperature (u_t) and thermostat program (W). Imagine house b starts with a cold temperature (x_0) and house a starts with a hot temperature (x'_0). With time and a reasonable thermostat program both temperature states (x_n, x'_n) will converge and the temperature state (x_n) will be determined solely by the temperature target.

2.2 Hyper-Parameters

- **Spectral radius** $\rho(W_{\text{res}})$: Let $W \in \mathbb{R}^{N\times N}$ be a square matrix with eigenvalues $\{\lambda_i\}_{i=1}^{N}$. The *spectral radius* of W is defined as

$$\rho(W) = \max_{1\le i\le N} |\lambda_i|.$$

  The spectral radius roughly controls the strength of recurrent feedback. In the common (heuristic) setting, choosing $\rho(W_{\text{res}}) < 1$ fosters the echo state property by making the reservoir dynamics contractive. Values near $\rho(W_{\text{res}}) \approx 1$ typically yield richer, longer-lasting transients while still often preserving the echo state property. When the reservoir is driven by sufficiently strong inputs (non zero inputs in-practice), even $\rho(W_{\text{res}}) > 1$ can remain effectively stable in practice because the nonlinearity (activation function)(e.g., tanh) reduces the effective gain of the recurrence, damping feedback loops. Lukoševičius, 2012

  A reservoir that does not satisfy the echo state property is not useful for reliable computation. Without the echo state property, the reservoir state is not uniquely determined by the input history; depending largely on the initial condition. In practice, this means that recurrent feedback dominates the dynamics: internal activity grows in magnitude and persists independently of the input; the effect of new inputs is overshadowed. As a result, the reservoir ceases to act

as a dynamical system and instead consistently gravitates towards magnitude increasing feedback loops.

- **Leak rate** α: The leak rate controls the intrinsic timescale of the reservoir by interpolating between the previous state and the newly computed activation. The effective recurrence is

$$x_{t+1} \approx \left[(1-\alpha)x_t + \alpha\tilde{x}_{t+1}\right],$$

  Smaller α increases the persistence of past inputs and extends temporal memory of the reservoir. Larger α causes the system to update more rapidly, reducing memory but increasing responsiveness. Thus, while the spectral radius controls the strength of recurrence, the leak rate controls how quickly that signal dissipates over time.

- **Input scaling** $\|W_{\text{in}}\|$: Input scaling determines the relative strength of external input compared to recurrent feedback. When input scaling is small, reservoir dynamics are dominated by recurrence, and the system primarily reflects its internal structure. Increasing input scaling drives neurons into the nonlinear regime of the activation function, increasing computational richness. However, excessive scaling saturates the activation function, reducing sensitivity to inputs and collapsing the effective dimensionality of the reservoir's state matrix.

- **Noise** ξ_t: Noise introduces stochastic perturbations to the reservoir state. Noise insertion has been shown to improve attractor stability in echo state networks by preventing trajectories from collapsing into undesired fixed points or oscillatory modes, though this comes at the cost of reduced prediction precision Jaeger, 2001. Because the present experiments focus on task-agnostic structural invariance rather than attractor training, noise is not included as a feature of our model.

Chapter 3

Background

3.1 The C. elegans Connectome

3.1.1 Connectome

A connectome is a comprehensive structural map of the neural wiring of an organism. It encodes which neurons are connected, the direction of said connection, and often additional biological metadata such as the number of synaptic terminals and the neurotransmitter associated with each connection.

In this work, we treat the connectome as the blueprint for a graph: each neuron is represented as a node, and each synapse as a directed, weighted edge. The connectome used in our study is sourced from the c302 projectGleeson et al., 2018, which is an initiative to model the neural network of the nematode *C. elegans*. The network is composed of several key components:

- **Neurons**: The database includes all 302 identified neurons in the *C. elegans* nervous system. Our model includes 299 neurons; CANL, CANR, VC6 are omitted (see Sec. 3.1.2).

- **Synaptic connections**: The c302 connectome model contains 3672 synaptic connections but 564 them are neuromuscular junctions; sending neurotransmitters to muscle cells and are not included in our graph. This leaves us with 3108 synaptic connections, categorized into various types based on the nature of the synapse, including chemical synapses and gap junctions. Specifically, we assign inhibitory synapses (GABAergic) negative weights (190 synapses) and excitatory synapses–acetylcholine (394 synapses) and glutamate(962)–positive weights. All remaining transmitter (1562 synapses) are assigned positive sign. This sign assignment is a *modeling prior* used to define an excitatory/inhibitory partition for the invariance experiments; it does not attempt to capture receptor-specific or context-dependent effects (e.g., cases where acetylcholine or glutamate can be inhibitory). Because our study analyzes how coarse sign structure and weight statistics affect task-agnostic reservoir metrics under hyperparameter sweeps, incorporating detailed synapse-type dynamics is outside scope. We retain the non-GABA synapses (rather than dropping them) to preserve the

observed network density and recurrence, which strongly influence reservoir dynamics and the metrics studied here.

The connections are represented as directed edges in the network, indicating the direction of signal transmission. Gap junctions are considered to be two distinct edges with independent weights for computational simplicity.

- **Connection count (proxy weights)**: The dataset reports only the number of terminal contacts that neuron i makes onto neuron j, not graded synaptic strengths. We therefore use that contact count c_{ij} as an initial weight, $w_{ij} = c_{ij}$. Treating contact number as a proxy weight is standard in *C. elegans* modeling Casal et al., 2020; Gleeson et al., 2018 and provides the biologically informed prior for our training.

3.1.2 Omission of CANL/CANR

Omission of CANL/CANR. In the c302 wiring used here, CANL and CANR form gap-junction contacts only with excretory tissues and have no neuronal synapses. Because our node set includes neurons and muscles but not excretory cells, including CANL/CANR would produce two isolated vertices (no neuronal in- or out-edges and no neuromuscular outputs). Additionally, VC6 is a forms an outgoing connection to a muscle, but has no incoming or outgoing synapses to or from other neurons. Under the dynamics in Sec. 3.1.3, isolated nodes are decoupled from the controller and cannot influence motor commands or reward; their inclusion/exclusion is therefore behaviorally invariant in this model.

3.1.3 Isolated nodes

Nodes with no incoming or outgoing synaptic connections are dynamically inert and therefore do not influence dynamics of the network. Formally, if neuron i satisfies $W_{ij} = 0$ and $W_{ji} = 0$ for all j, then its state does not contribute to the state update of any other neuron. Consequently, adding or removing such isolated nodes does not alter the dynamics of the remaining network.

This condition applies to CANL, CANR, and VC6 in the c302-derived graph, as their known synaptic contacts are exclusively with excretory (CANL, CANR) and vulval muscles (VC6), which are omitted from the modeled neuron set. Consequently, these neurons are isolated within the weight matrix and do not influence the reservoir dynamics.

3.1.4 Collective dynamics of small-world networks

The small-world phenomenon refers to the observation that the distance between two randomly chosen nodes in a network is surprisingly short, even when the network contains many nodes Barabási, 2016. Small-world networks are characterized by the coexistence of two structural properties: a high clustering coefficient and a short characteristic path length.

Formally, let $\overline{L}$ denote the average path length and $\overline{C}$ denote the average clustering coefficient of the network. To quantify small-world structure, these quantities are typically normalized relative to an equivalent random network with the same number of nodes and edges, denoted $\overline{L}_{\text{rand}}$ and $\overline{C}_{\text{rand}}$, respectively. The small-worldness metric is then defined as

$$S = \frac{\overline{C}/\overline{C}_{\text{rand}}}{\overline{L}/\overline{L}_{\text{rand}}}.$$

A network is considered small-world when $S > 1$, indicating that it maintains higher clustering than a random network while preserving similarly short path lengths.Humphries and Gurney, 2008

Watts and Strogatz introduced a generative model that demonstrates how small-world structure can emerge from rewiringWatts and Strogatz, 1998. The model begins with a regular network, in which each node is connected to its k nearest neighbors. Each edge is then rewired to a randomly selected node with probability p. When $p = 0$, the network is completely regular, exhibiting high clustering but long path lengths. When $p = 1$, the network becomes fully random, exhibiting short path lengths but low clustering. At intermediate rewiring probabilities, typically around $p \approx 0.1$, the network simultaneously exhibits high clustering and short path lengths, producing small-world networks.

3.1.5 A small-world topology enhances the echo state property and signal propagation in reservoir computing

This paperKawai et al., 2019 lays the ground work for our study. They use the Watts/StrogatzWatts and Strogatz, 1998 small worldness model in their experiments, they vary p and the spectral radius $\rho(W_{res})$ and demonstrate small worldness ($p = 0.1$) leads to a greater $\sum_{\rho(W_{res})}$ MC when input and output nodes were segregated (input nodes are close together and on the opposite side of the output nodes that are close together), rather than randomly selected (without overlaps). While, the networks with small worldness preformed the strongest over the largest variety of spectral radii, the traditional [1] network ($p = 1$) had the highest memory capacity, reached at a spectral radius just less than one. This paper is the seed idea for this work as it shows a biologically inspired architecture exhibit comparatively stable performance across spectral radii compared to a randomly generated architecture.

3.2 Task Agnostic Network Statistics

3.2.1 Motivation

What makes a given neural circuit computationally powerful? More precisely, how can two network architectures be compared across a suite of tasks without explicitly training them on those tasks? In reservoir computing, this question is addressed

[1] traditional refers to theory in compute reservoirs that a random network is always the best, as it has the most diverse set of dynamics

through task-agnostic statistics that quantify intrinsic dynamical properties of the network. Measures such as information processing capacity (IPC), kernel rank (KR), generalization rank (GR), and memory capacity (MC) characterize complementary functional dimensions, including nonlinearity, separability, generalization, and fading memory. Additionally, these statistics are widely used as benchmarking tools for reservoir networks Pilati et al., 2026.

3.2.2 Kernel Rank

One could also drop different sized stones sequentially at different places in the pool. They could do this experiment multiple times; observing how distinct the resulting wave states are. Kernel rank is a measure of how distinctly the pool responds to different sets of inputs; if many input sequences lead to the same wave patterns, then the kernel rank will be low. If those input sequences instead lead(map) to distinct wave patterns then the kernel rank will be high.

Kernel rank measures how effectively inputs are projected into a high-dimensional state space so they become linearly separable by a readout W^{out}, serving as a measurable proxy for kernel quality and the diversity of nonlinear transformations created by the reservoirStepney, 2024. Higher is better for this metric. Specifically it is a measure of the rank of the $n \times m$ matrix who's columns are the internal state $(x_{u_i}(t_0))$ at resulting at some fixed time t_0 for the preceding input stream u_i. Where n is the number of units in the circuit, m is number of input streams, $(x_{u_i}(t_0))$ is the internal state at time t_0, we have m different input streams u_i. This is the original definitionMaass et al., 2004, we will use a modified version of this also found in literature Stepney, 2024. In our implementation, we use the state-matrix variant common in benchmarking: we drive the reservoir with a single i.i.d. input sequence u_t—rather than m input streams—and collect the post-washout state matrix $X \in R^{T \times N}$ where each row is the reservoir state at each time step. Kernel Rank is computed using effective rank Roy and Vetterli, 2007. To compute effective rank, first we must center our distribution and do a singular value decomposition

$$X' = \text{SVDVALS}(X - \bar{X})$$

$$X''_i = \max(\epsilon, X'_i)$$

$$p_i = \frac{X''_i}{\sum_{k=0}^{N} X''_k}$$

$$H = -\sum_{i=0}^{N} (p \log p)$$

$$\text{erank} = e^H$$

3.2.3 Generalization Rank

Returning to the pool, suppose we wanted to observe how much the waves patterns change if we used slightly larger or smaller rocks. We could alter the size of our

rocks slightly and drop them at slightly different places. By comparing the resulting wave pattern to the wave pattern of the unaltered rocks, we can measure how many independent wave patterns the pool produces from these nearly identical inputs. If small changes in the stones lead to many distinct wave patterns, the pool is highly sensitive and maps to a high-dimensional set of states; if small changes lead to similar wave patterns, the pool generalizes and collapses to a lower-dimensional set of states. This is measured by the Generalization Rank.

Generalization Rank (GR) quantifies the sensitivity of reservoir states to small perturbations in the input. It is computed as the effective rank of the difference between state trajectories driven by clean and noise-perturbed inputs. Lower GR indicates that input perturbations induce lower-dimensional deviations in state space, reflecting greater robustness and reduced sensitivity to noise.

$$u_t \overset{\text{i.i.d.}}{\sim} \mathcal{U}(-1, 1)$$

$$u'_t = u_t + \kappa\, \varepsilon_t, \qquad \varepsilon_t \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1).$$

with κ in our version being set by 0.01.
then we run the network with the perurbed u', this network will have states X' and with the unperturbed u, with states X.

$$\text{Generalization Rank} = \text{erank}(X' - X)$$

if our network generalizes well, small amounts of noise on the input will not change the internal state after a set washout period.

3.2.4 Memory Capacity

Looking at the pool a human seeing the waves but not the stone would be able to roughly estimate or *reconstruct* the size of said stone (assuming all stones were dropped from the same height). If multiple stones were sequentially dropped into the pool, it would be harder to reconstruct the sizes of said stones, but it would still be computationally possible. The ability to reconstruct the size of stones solely by observing the state of the pool–the waves–is called memory capacity. It is a measure of how much information of the input remains in our pool after a given number of time steps. To illustrate this picture this example, you have 3 stones of different sizes–small, medium, and large–suppose every second for 10 seconds you drop a different stone into the water, and at the 10 seconds you tried to reconstruct the size of every stone dropped in by looking at the waves– the first stone would be challenging to reconstruct as after 10 seconds its waves would both have reduced in size and would have mixed with other waves. In reservoir computing this metric is called Memory capacity.

Memory capacity, introduced by Jaeger Jaeger, 2002, is a measure of the amount of linear memory a reservoir contains. Given an input $u(t) \in U[-1, 1]$, the reservoir's memory capacity is determined by its aiblity to reproduce this input at at varying delays (k). Memory capacity is originally defined by Jaeger as

$$\mathrm{MC}_k = \frac{\big(\mathrm{cov}(u(n-k), \hat{u}_k(n))\big)^2}{\mathrm{var}(u)\,\mathrm{var}(\hat{u}_k)}$$

$\mathrm{MC} = \sum_{k=1}^{\infty} \mathrm{MC}_k$.

MC_k measures the correlation between the predicted signal and the original signal or how well the network "remembers" the original signal at time k. Summing over k gives us how well the network can remember a signal over all delays k or how good the temporal memory is. As k increases the MC_k typically decreases, this is intuitive, as we are in a network that has the echo state property (the state of the network is determined by its input history and not by its initial state). Hence we try to recreate the input u_{t-k} from the state x_t, but because our network has the echo state property, the state, x_{t-k} resulting from the input u_{t-k} loses influence over x_t as k grows. Jaeger also shows that for i.i.d. (independent and identically distributed) random input, $MC \leq N$ Jaeger, 2002; Stepney, 2024. Their equation is the Pearson correlation coefficient squared, which is equal to R-squared. Hence we will write our equation as

$$MC = \sum_{d=1}^{D_{\max}} \max(0.0, R^2_{\text{score}}(u^{\text{test}}_{\{t_i\}_{i=0}^{d}}, X^{test}_{\{t_i\}_{i=d}^{N}} \text{Ridge_Regression}(X^{\text{train}}_{\{t_i\}_{i=d}^{N}}, u^{\text{train}}_{\{t_i\}_{i=0}^{d}})))$$

where $D_{\max}$ is the user specified max delay (we used a value of 300). This both differs from the formal definition, we choose to not sum to infinity because this is computationally unfeasible and because MC_k degrades as $k \to \infty$ hence summing MC_k with high k is equivalent to adding noise. We also use a higher $D_{\max}$ than the one used in IPC experiments because memory capacity is a simpler task: it tests only linear retention, whereas IPC also requires nonlinear expansion.

3.2.5 Information Processing Capacity

Returning to our pool, suppose we are no longer interested in recovering the size of individual stones, but instead in recovering combinations of stones. When multiple stones are dropped into the pool, their waves do not simply add; they interfere, overlap, and mix. The resulting wave pattern contains not only information about each stone individually, but also information about their interactions. Information Processing Capacity measures how many independent combinations of past stones simultaneously exist in the current wave pattern.

Information capacity is a measure of a dynamical systems ability to recreate functions of its input from the current state of the system. We start with a uniformly sampled noise sequence of length t on the bounds [1,-1]

$$u_t \overset{\text{i.i.d.}}{\sim} \mathcal{U}(0,1)$$

First we rescale the input sequence u to the range for -1–1 for both test and trian data

$$\frac{2(u - \min(u))}{\max(u) - \min(u) + \epsilon} - 1.0$$

importantly the mean of u after this rescaling is ≈ 0. Let $a, b = \min(u), \max(u)$. We will show $\mathbb{E}(\tilde{u}) = 0$

$$\mathbb{E}(\tilde{u}) = \frac{2(u-a)}{b-a+\epsilon} - 1.0 \tag{3.1}$$

$$\frac{2u}{b-a+\epsilon} - \frac{2a}{b-a+\epsilon} - 1.0 \tag{3.2}$$

$$\frac{2u}{b-a+\epsilon} + \frac{-2a}{b-a+\epsilon} - \frac{b-a+\epsilon}{b-a+\epsilon} \tag{3.3}$$

$$\frac{2u}{b-a+\epsilon} + \frac{-b-a-\epsilon}{b-a+\epsilon} \tag{3.4}$$

$$\mathbb{E}(\tilde{u}) = \frac{a+b}{2} \tag{3.5}$$

$$\frac{2}{b-a+\epsilon}\frac{a+b}{2} + \frac{-b-a-\epsilon}{b-a+\epsilon} \tag{3.6}$$

$$\frac{2(a+b)}{2(b-a+\epsilon)} + \frac{-b-a-\epsilon}{b-a+\epsilon} \tag{3.7}$$

$$\frac{2(a+b)}{2(b-a+\epsilon)} + \frac{-2b-2a-2\epsilon}{2(b-a+\epsilon)} \tag{3.8}$$

$$\frac{-2\epsilon}{2(b-a+\epsilon)} \tag{3.9}$$

using linearity and multiplicity of expectation. because epsilon ≈ 0 then $\mathbb{E}(\tilde{u}) = 0$

$$R^2_{\text{score}}(Y, \hat{Y}) = 1 - \frac{\sum_{i=1}^{N}(Y_i - \hat{Y}_i)^2}{\sum_{i=1}^{N}(Y_i - \bar{Y})^2 + \epsilon}$$

= where y is the true input signal, $\hat{y}$ is the predicted input signal, and ϵ i a tiny value to ensure that we do not divide by 0.
then we do matrix ridge regression

$$\text{Ridge_Regularization}(X_{\text{train}}, Y_{\text{train}}, \xi) = ((X')^T X' + \xi I)^{-1}(X')^T Y'$$

Where $X' = \frac{X-\bar{X}}{\text{std}(X)+\epsilon}$ ξ is a user specified parameter denoting the strength of L2 regression I is the identity matrix with dimension N, N note: in our implementation we use $Y' = Y$ the typical equation has Y = $\frac{Y-\bar{Y}}{\text{std}(Y)+\epsilon}$ we can skip this for a computational speedup by knowing that $\bar{Y} \approx 0$ hence $Y' = \frac{Y}{\text{std}(Y)+\epsilon}$, however this can be written as $Y' = aY$ with $a = \frac{1}{\text{std}(Y)+\epsilon}$ we can just multiply the weight matrix found from ridge regularization by a. We define information processing capacity (IPC) as

$$\text{IPC} = \sum_{d=1}^{D_{\max}} \sum_{k=1}^{K_{\max}} \max\big(0.0,\ R^2_{\text{score}}(Y, \hat{Y})\big),$$

where

$$Y = P_k\Big(u^{\text{test}}_{\{t_i\}_{i=0}^{d}}\Big), \qquad \hat{Y} = X^{\text{test}}_{\{t_i\}_{i=d}^{N}} \text{Ridge_Regression}\Big(X^{\text{train}}_{\{t_i\}_{i=d}^{N}},\ P_k\Big(u^{\text{train}}_{\{t_i\}_{i=0}^{d}}\Big)\Big).$$

Here $D_{\max}$ and $K_{\max}$ (set to 50 and 3, respectively) are the maximum delay and maximum polynomial degree. We cap $D_{\max} = 50$ for two reasons: (i) evaluating all (k, d) pairs requires $K_{\max} D_{\max}$ ridge fits per model, which is expensive at our scale; and (ii) reconstructing higher-order (polynomial) functions of the input is substantially harder than linear reconstruction, large delays contributes less as k increases.

In the literature, IPC is often summed over all Legendre polynomial degrees. We instead truncate at $K_{\max} = 3$ because contributions degrade with increasing k, and because our experiments require tens of thousands of IPC evaluations. Typical cutoffs are $K_{\max} = 3$ or 5; we use 3 for runtime.

Notation.

- $X^{\text{train}}_{\{t_i\}_{i=d}^{N}}$: reservoir states on training data from timestep d to the end.
- $X^{\text{test}}_{\{t_i\}_{i=d}^{N}}$: reservoir states on test data from timestep d to the end.
- $u^{\text{train}}_{\{t_i\}_{i=0}^{d}}$: training inputs from the start up to delay d.
- $u^{\text{test}}_{\{t_i\}_{i=0}^{d}}$: test inputs from the start up to delay d.
- P_k: Legendre polynomial of degree k.

3.3 Invariance across Hyper-parameters

Variance measures the degree of spread in a data set from its mean value. In this study we are measuring variance of key network statistics (GR,KR,IPC,MC) under hyperparameter change. It is important to note that while the values of these network statistics are important, i.e. a high IPC being an indicator of a strong network architecture, we are investigating how these values change under hyperparameter change compared between different model architecture, rather than the values themselves. Hence we will not measure variance but rather we will measure coefficient of variation, as this provides a measure of variance across different scales.

$$CV = \frac{\text{std}(a)}{\bar{a}}$$

Where a is the set of values of the network statistic we are measuring, i.e. IPC, MC, GR, or KR, under a set of differing hyperparameter values. It is desirable to have a low CV, as this indicates that the network statistic is invariant to hyperparameter change, meaning the network can be trained with a wide range of hyper-parameters without changing the network's ability to perform a given task.

3.3.1 Hyperparameter Sweeps

We evaluated network performance under perturbations of spectral radius, leak rate, and input scaling. To compute our coefficient of variation (CV) metric, each trial

consisted of evaluating the network across all combinations of the following hyperparameters:

- Spectral radius: $sr \in \{0.6, 0.8, 0.95, 1.05\}$
- Leak rate: leak $\in \{0.6, 0.8, 1.0\}$
- Input scaling: input scaling $\in \{0.1, 0.5, 1.0, 1.5\}$

For each trial, the CV was computed across all hyperparameter configurations. In analyses requiring a single summary value per condition, CV values were averaged across trials.

Chapter 4

Methods

4.1 Network Architectures

4.1.1 Celegan connectome

This uses the biological network and biological weights. Used mostly as a comparison against other methods.

$$w_{ij} = w_{ij}^{cel},$$

4.1.2 Connection-Shuffle

We do a connection shuffle on the Celegan connectome; preserving single-point connectivity. For our connection shuffle we randomly select with distinct nodes, a, b, c, d, that have edges between nodes a, b and c, d. Then we rewire the edges such that the edge between a, b with edge weight w_1 is removed and replaced with an edge a, d with edge weight w_1. Additionally, the edge c, d with edge weight w_2 is removed and replaced with an edge c, b with weight w_2. This preserves in/out-node degree, while shuffling the neighbors of each node.

4.1.3 Weight-Shuffle

Fix all synaptic connections but shuffle their weights. This is achieved by taking getting the indices of the non zero weights of the weight matrix (NZ). Then we copy the weights that these indices point to, to a new matrix, W_{nz}. Finally we shuffle we shuffle W_{nz} using

```
numpy.random.shuffle
```

and reassign the shuffled weights back to the original indices. The probability that a given connection will change sign is 11.47%. The expected number of connections

that will change signs is 356.769.

$$N_{\text{neg}} = \text{number of negative edge weights} \tag{4.1}$$
$$N = \text{number of weights} \tag{4.2}$$
$$P_{\text{neg}} = P(\text{negative}) = \frac{N_{\text{neg}}}{N} \tag{4.3}$$
$$P(\text{neg} \to \text{pos}) = P(\text{positive})\, P(\text{negative}) \tag{4.4}$$
$$= (1 - P_{\text{neg}})P_{\text{neg}} \tag{4.5}$$
$$P(\text{pos} \to \text{neg}) = P(\text{negative})\, P(\text{positive}) \tag{4.6}$$
$$= P_{\text{neg}}(1 - P_{\text{neg}}) \tag{4.7}$$
$$P(\text{swap}) = P(\text{pos} \to \text{neg}) + P(\text{neg} \to \text{pos}) \tag{4.8}$$
$$= (1 - P_{\text{neg}})P_{\text{neg}} + P_{\text{neg}}(1 - P_{\text{neg}}) \tag{4.9}$$
$$= 2P_{\text{neg}}(1 - P_{\text{neg}}) \tag{4.10}$$
$$\mathbb{E}[\text{swap}] = N\, P(\text{swap}) \tag{4.11}$$

$$P_{neg} = \frac{190}{3108} \tag{4.12}$$
$$P(\text{swap}) = 2(\frac{190}{3108})(1 - \frac{190}{3108}) = 0.1147 \tag{4.13}$$
$$\mathbb{E}(\text{swaps}) = 2(\frac{190}{3108})(1 - \frac{190}{3108}) \times 3108 = 356.7696 \tag{4.14}$$

4.1.4 Connection-shuffle with Weight-Shuffle

First shuffle connections the connections of the celegan connectome using 4.1.2, then shuffle the weights of the resulting weight matrix using 4.1.3. This method is mostly used as a control to show the effects of an additional weight shuffle after a connection shuffle.

4.1.5 C. elegans connections with Gaussian Weights

Keep all fix all synaptic connections but resample their weights from a gaussian distribution centered at 0 with spread 1. This is achieved by taking getting the indices of the non zero weights of the weight matrix (NZ). Then creating a new weight vector, V_{new}, of size $NZ \times 1$ with each entry being randomly selected from a gaussian distribution centered at 0 with spread 1. Then we reassign the weights at W_{nz} to V_{new}

$$w'_{ij} = \mathcal{N}(0, 1),$$

4.1.6 Celegan matched Erdos Renyl Model With Gaussian Weights

Random edges with the name number of edges as in the celegan connectome, with weights sampled from a gaussian distribution. Create a random graph using the

Erdos-Renyl (ER) model. This model include any given connection in the graph with probability P. We create a graph of size N then randomly add edges with the probability of each edge being included in the graph equal to 0.1. We then randomly remove connections until we have the same number connections as the celegan connectome, these connections have weight values set to 0.. Then we assign weights, to these edges, drawn from a guassian distribution (center = 0, spread = 1) to the remaining edges.

$$W_{ij} \sim \mathcal{N}(0, 1),$$

4.1.7 Celegan matched Watts–Strogatz Model With Gaussian Weights

The Watts-Strogatz model starts with a regular graph, with each node connected to k other nodes. Then each edge is selected for a subset with probability p (we used a p of 0.1). Edges in this subset, (u, v) are rewired to a random vertex $(u, v\prime)$ with probability p. We then randomly remove connections with until we have the same number of connections as the celegan connectome, these connections have weight values set to 0. Then we assign weights, to these edges, drawn from a gaussian distribution (center = 0, spread = 1) to the remaining edges.

$$W_{ij} \sim \mathcal{N}(0, 1),$$

4.1.8 Celegan architecture with Gaussian weights using original Connection Signs

This uses the biological network and associated edge signs (keeps the neurotransmitter constant). Then it reassigns the edge weights to random weights sampled from a gaussian distribution centered at 0 with spread 1.

$$w'_{ij} = \text{sign}(w_{ij})(\mathcal{N}(0, 1)),$$

4.1.9 Celegan architecture with uniform weights using original Connection Signs

This uses the biological network and associated edge signs (keeps the neurotransmitter constant). Then it reassigns the edge weights to random weights sampled from a uniform distribution with low = 0.0 and high = 1.0.

$$w'_{ij} = \text{sign}(w_{ij})(\mathcal{U}(0, 1)),$$

4.1.10 Celegan architecture with sampled weights using original Connection Signs

This uses the biological network and associated edge signs (keeps the neurotransmitter constant). Then it reassigns the edge weights to random weights sampled the celegan

weight distribution. The negative weights are sampled from the negative weights in the celegan connectome. The positive weights are sampled from the positive weights in the connectome.

$$\mathcal{D}_{\text{CE}} := \{\, W_{ij}^{\text{CE}} \mid W_{ij}^{\text{CE}} \neq 0 \,\}$$
$$w'_{ij} = \text{sign}(w_{ij})(\mathcal{U}(\mathcal{D}_{\text{CE}})),$$

4.1.11 Celegan architecture with +1/-1 weights using original Connection Signs

This uses the biological network and associated edge signs (keeps the neurotransmitter constant). Then it sets the weight of every connection to 1 or -1 depending on the neurotransmitter.

$$w'_{ij} = \begin{cases} \text{sign}(w_{ij}) & \text{if } w_{ij} \neq 0 \\ 0 & \text{if } w_{ij} = 0 \end{cases}$$

4.1.12 Celegan architecture with +1/-1 weights using original Connection Sign balance with the signs shuffled

This uses the biological network and associated edge signs (keeps the neurotransmitter constant). Then it sets the weight of every connection to 1 or -1 depending on the neurotransmitter. Then I count the number of -1 signs (num_{neg}), take the absolute value of the weight matrix and randomly assign num_{neg} weights to be -1.

4.2 Trial definition

A single trial of a model consists of:

1. a reservoir instantiation for each hyperparameter combination
2. a performance evaluation for each hyperparameter combination. Using the following parameters
   - **Sequence Length** = 2000
   - **Washout time steps** = 500
   - **Perturb STD** = 0.01 (for calculating GR)
   - **Train time steps** = 1500
   - **Test time steps** = 500
   - **Memory capacity Max delay**
   - **Information processing capacity max delay**
   - **Information processing capacity max polynomial order**
   - **Ridge Alpha** = 0.0001 (controls the strength of L2 regularization for ridge regression)

The coefficient of variation (CV) is computed across hyperparameter combinations within a trial. Each trial uses a single fixed input sequence shared across all hyperparameter combinations to ensure comparability. Reservoir instantiations are independent across hyperparameter combinations: for each combination, the reservoir is reinitialized with a unique seed.

Number of trials for histogram experiment 5.5: 500 per model. Total metric evaluations per model: Trials $\times$ hyperparameter grid $= 500 \times 4 \times 3 \times 4 = 24,000$. Number of trials for 5.4, 5.3: 250 per point. Total metric evaluations per point: $250 \times 4 \times 3 \times 4 = 12,000$. Number of trials for 5.1 and 5.1: 50 per point. Total metric evaluations per point: $50 \times 4 \times 3 \times 4 = 2,400$.

4.3 Equivalence testing

We use two one-sided tests (TOST) to test whether the difference in mean CV between architectures lies within equivalence bounds. Equivalence bounds are defined as:

$$\Delta = \eta \cdot \text{median}(CV_{\text{all}})$$

We test:

$$H_0 : |\mu_1 - \mu_2| \geq \Delta$$

$$H_1 : |\mu_1 - \mu_2| < \Delta$$

Equivalence is concluded if both one-sided tests reject H_0 at $\eta = 0.05$.

Chapter 5

Results

The *C. elegans* connectome exhibits consistently low dispersion across hyperparameter sweeps, indicating that its computational properties remain stable under variation in spectral radius, leak rate, and input scaling. This is visible in Fig. 5.5, where the connectome's coefficient-of-variation (CV) distributions for MC, IPC, KR, and GR are concentrated toward lower values relative to several perturbed architectures.

To identify which structural features contribute to this invariance, we systematically perturbed connectivity, sign structure, and weight magnitude distributions. Preserving the biological excitatory/inhibitory synapse sign while altering weight magnitudes produced mixed effects that depended on the metric. When magnitudes were drawn from a Gaussian distribution while preserving sign (`local-sign`), equivalence to the biological connectome was supported for memory-related metrics (MC: $p = 1.381 \times 10^{-4}$; IPC: $p = 7.66 \times 10^{-3}$), but not for kernel-based metrics (KR and GR; Table 5.1). Suggesting that sign structure, and architecture combine to preserve the hyperparameter invariance of memory capacity and information processing capacity within the chosen equivalence bound.

Furthermore, when all magnitudes were set to uniform values under preserved biological sign (`local-sign+flat`), invariance was equivalent to the biological connectome for the MC and IPC metrics (MC: $p = 2.694 \times 10^{-37}$; IPC: $p = 9.865 \times 10^{-30}$), however it as well was not significantly within the tost bound of the biological connectome for kernel based metrics. Conversely, the binary-weight model (`local-sign+binary`) preserved invariance for MC,GR,KR (MC: $p = 9.13 \times 10^{-4}$), KR: $p = 3.46 \times 10^{-46}$; GR: $p = 6.00 \times 10^{-58}$) but failed for (IPC: $p = 0.203$, as it was actually more invariant than the biological connectome. These results suggest that preserving sign structure, and edge structure preserve invariance across varied regimes. Yet, this conclusion lacks a dimension, it is not just sign structure, and edge structure but the interaction between sign and edge structure with the distribution of weight magnitudes determines which computational properties remain invariant. In this paper we seek to elucidate this relationship.

First: where connections go is unimportant for task invariance, so long as the single point degree matches the celegan connectome. This is evidenced by the Weight shuffle model being as invariant as the connection shuffle model and the connection shuffle + weight shuffle model. We claim that the invariance increase is caused by

the weight shuffle inevitably caused by the connection shuffle and not the connection shuffle itself. This is because if the connection shuffle provided additional invariance beyond that caused by the weight shuffle we would see an invariance gain, when we also shuffled the weights in the dual model. its not the ordering but the distribution

5.1 Weight magnitude distribution causally Both modulates performance and hyperparameter sensitivity

We next isolate the effect of weight magnitude distribution while holding connectivity and sign structure fixed. Specifically, we compare the local sign–preserved connectome with binary weights (± 1) to an otherwise identical model in which nonzero weights are perturbed by additive positive noise drawn from a stacked half-normal distribution. Formally, for each nonzero entry $w_{ij} \in \{\pm 1\}$, we construct

$$w'_{ij} = \mathrm{sign}(w_{ij})(1 + \lambda\,|\mathcal{N}(0,1)|),$$

where λ controls the magnitude distribution while preserving the original sparsity pattern and sign structure. This procedure alters only the distribution of weight magnitudes, allowing causal attribution of any observed changes in computational properties to magnitude alone.

Figures 5.1 and 5.2 show the resulting relationship between weight magnitude statistics, performance, and hyperparameter sensitivity. As λ increases, the mean absolute weight value increases monotonically. This increase is accompanied by a rise in both mean computational performance and the coefficient of variation. These results demonstrate that weight magnitude distribution independently governs computational performance, and sensitivity of that performance to hyperparameter variation. Notably, the increase in mean weight magnitude exhibits a clear saturation regime at large λ. This behavior is consistent with spectral radius normalization: as the overall scale of weights increases, subsequent normalization compresses differences in magnitude, limiting further increases in the mean effective weight scale. Thus, while increasing λ initially expands the dynamical regime and improves computational capacity, normalization imposes an upper bound on the effective magnitude scale.

5.2 Global neurotransmitter balance, not synapse-specific sign placement, determines invariance

We next test whether invariance depends on the precise placement of excitatory and inhibitory synapses, or instead on the global balance of neurotransmitter signs. To isolate this factor, we compare the Local sign–preserved binary model to a Global sign model in which the overall proportion of excitatory and inhibitory weights is preserved, but their locations are randomized.

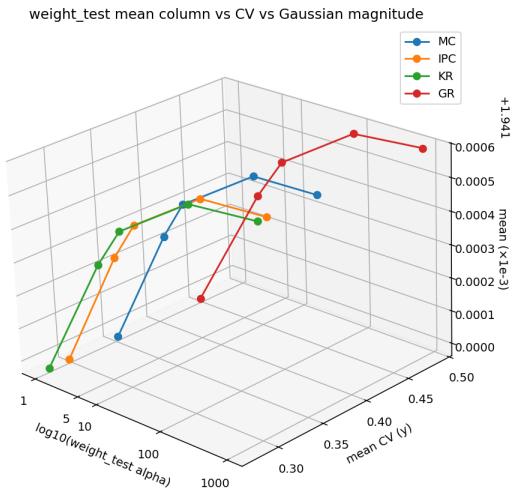

Figure 5.1: Mean coefficient of variation (z) and mean weight magnitude (y) as a function of perturbation magnitude Λ (x-axis, log scale). Nonzero binary weights are perturbed according to $w'_{ij} = \text{sign}(w_{ij})(1 + \Lambda|\mathcal{N}(0,1)|)$, preserving connectivity and sign structure. Increasing Λ increases both the mean weight magnitude and the coefficient of variation across hyperparameter settings.

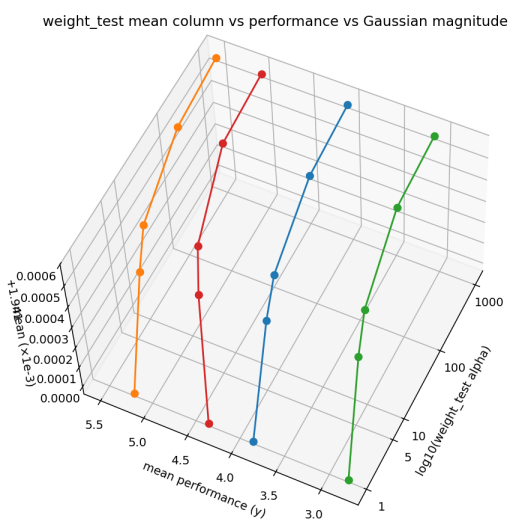

Figure 5.2: Mean computational performance (z) and mean weight magnitude (y) as a function of perturbation magnitude Λ (x-axis, log scale), with connectivity and sign structure fixed. Increasing weight magnitude distribution scale increases achievable performance, followed by saturation consistent with spectral radius normalization.

We assess equivalence using the two one-sided tests (TOST) procedure with predefined equivalence bounds. The Global sign model falls significantly within the equivalence bounds relative to the Local sign–preserved binary model across all invariance metrics. This establishes statistical equivalence between the two conditions.

This result demonstrates that the specific spatial placement of excitatory and inhibitory synapses is not a causal determinant of task-agnostic computational invariance. Instead, invariance depends primarily on the global balance of excitation and inhibition. Preserving the overall excitatory/inhibitory ratio is sufficient to maintain invariant computational properties, even when the precise locations of synapse signs are randomized.

These findings indicate that invariance is governed by global dynamical constraints imposed by sign balance, rather than by fine-scale structural placement of individual synapse polarities.

5.3 Global neurotransmitter balance is a causal determinant of invariance

We next test whether global neurotransmitter balance, independent of weight distribution and weight structure, causally determines task-agnostic invariance. To isolate this factor, we compare two models with identical connectivity and identical weight magnitude distributions, differing only in whether the original synaptic sign structure is preserved. Specifically, we compare the *C. elegans* + $\mathcal{N}(0,1)$ model, in which weights are resampled from a zero-centered normal distribution without preserving original signs, to the *Local sign–preserved* + $\mathcal{N}(0,1)$ model, in which weight magnitudes are drawn from the same distribution but original synapse signs are maintained.

Despite identical connectivity and magnitude statistics, these two models exhibit markedly different invariance properties. The *C. elegans* + $\mathcal{N}(0,1)$ model falls within the high-variance cluster, exhibiting some of the lowest invariance observed across all tested models. In contrast, the sign-preserved model lies within the low-variance cluster, closely matching the invariance properties of the original *C. elegans* connectome. Because synapse sign preservation is the only difference between these models, this establishes global neurotransmitter balance as a causal determinant of task-agnostic invariance.

To further isolate the effect of neurotransmitter balance, we performed a controlled sign-flipping experiment in which a fraction f of randomly selected synapses had their signs flipped while preserving connectivity and weight magnitudes (Figures 5.3 and 5.4). This procedure continuously perturbs the global excitatory/inhibitory balance while holding all other structural properties fixed.

At $f = 0.0$, corresponding to the original *C. elegans* connectome, the network exhibits maximal invariance but reduced mean computational performance. As the fraction of flipped synapses increases, performance initially improves while invariance decreases. The highest mean computational performance occurs at a balanced excitatory/inhibitory regime, consistent with prior observations that weight distribu-

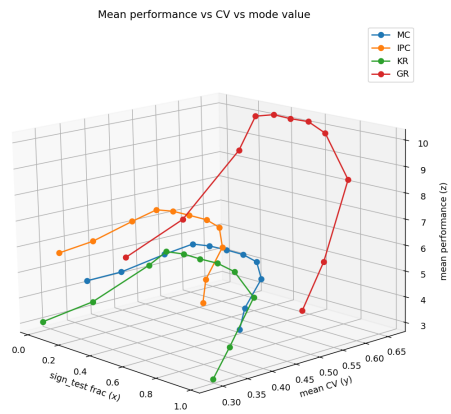

Figure 5.3: Mean computational metrics as a function of the fraction of synapse signs flipped from the original connectome using binary weights. Increasing sign randomization shifts the network from a low-variance, invariant regime toward a higher-performance but less invariant regime. $N = 250$ for each sign fraction

tions centered at zero promote richer dynamical regimes. However, this performance increase is accompanied by substantially increased sensitivity to hyperparameter variation.

These results reveal a tradeoff imposed by neurotransmitter balance: networks with highly asymmetric sign structure exhibit reduced computational capacity but greater invariance and stability, whereas networks approaching a balanced excitatory/inhibitory regime achieve higher performance at the cost of increased variability. This explains the behavior of the *C. elegans* + $\mathcal{N}(0,1)$ model, whose zero-centered magnitude distribution disrupts the original biological sign asymmetry, shifting the network toward a high-performance but low-invariance regime.

These findings establish global neurotransmitter balance as a primary determinant of task-agnostic invariance, independent of magnitude distribution. The biological connectome appears tuned toward a regime that sacrifices maximal computational capacity in favor of robust, invariant computation across hyperparameter variation.

Architectures that preserved empirical weight statistics without preserving sign structure (`cel-sample`) produced dispersion patterns similar to those of sign-preserving empirical sampling (`local-sign+sample`), indicating that weight magnitude distributions contribute independently to invariance.

Equivalence testing using TOST (Table 5.1) further clarifies these relationships. We used TOST equivalence testing on the mean dispersion (CV) between each architecture and the biological connectome. The equivalence margin was set to $\Delta = |0.05(\text{median}(\text{dispersion}))|$ computed per metric accross all runs. We concluded equivalence when both one-sided tests rejected the null that the mean difference lay outside $[-\Delta, +\Delta]$ Thus, small p-values indicate that the dispersion difference is statistically confined within the predefined practical tolerance. Under this criterion, only specific sign-preserving perturbations satisfied equivalence for selected metrics, while most

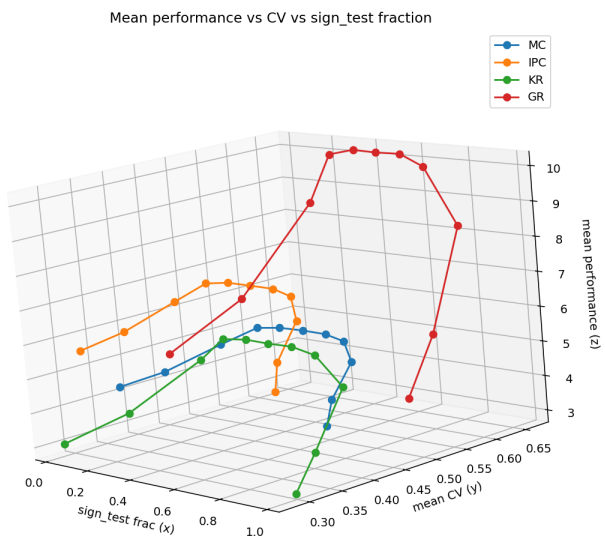

Figure 5.4: Mean computational performance and invariance metrics as a function of the fraction of flipped synapse signs. The original biological sign structure ($f = 0$) exhibits maximal invariance, while increasing sign randomization improves performance but increases variability. $N = 250$ for each sign fraction

architectures—including connection shuffles, weight shuffles, and random graph controls—did not satisfy equivalence across all metrics simultaneously. Failure to conclude equivalence (large p) does not imply non-equivalence; it indicates insufficient evidence that the mean difference is within $\pm\Delta$ under the chosen margin and sample size. 1000 obersvations for each comparison.

Architecture color legend. Blue: empirical *C. elegans* connectome with biological weights. Dark green: empirical topology with Gaussian-resampled weights. Light green: Erdős–Rényi random graph with Gaussian weights. Neon green: Watts–Strogatz small-world graph ($p = 0.1$) with Gaussian weights. Red: degree-preserving connection shuffle followed by weight shuffle. Teal: empirical topology and signs preserved, magnitudes Gaussian. Orange: weight shuffle only. Salmon: weights resampled (with replacement) from empirical distribution. Light orange: connection shuffle only. Deep blue: empirical topology and signs preserved, magnitudes uniform. Purple: empirical topology and signs preserved, magnitudes sampled by sign. Gray: keep the same balance of excitatory–inhibitory synapses but shuffle the neurotransmitter attached to each synapse. Light cyan: empirical topology and signs preserved, weights all set to $\{\pm 1\}$.

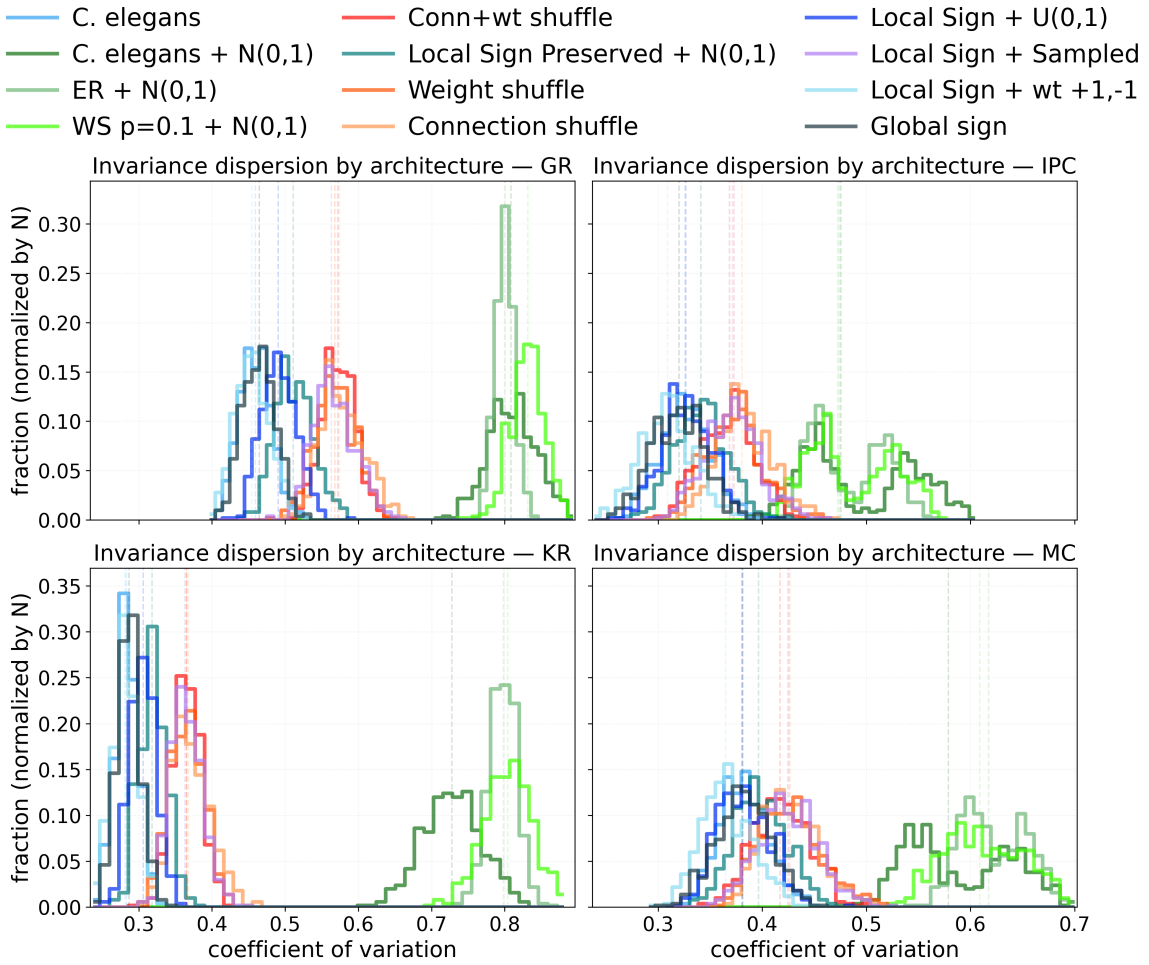

Figure 5.5: Invariance dispersion across network architectures, quantified as the coefficient of variation (CV) of each task-agnostic metric over hyperparameter sweeps. Panels show GR, IPC, KR, and MC. Curves are normalized histograms of CV (bin counts divided by the number of runs in that condition); dashed vertical lines mark medians. Lower CV indicates greater invariance to hyperparameter choice.

Table 5.1: Pairwise equivalence tests (TOST) across modes within each task agnostic performance metric.

Metric	Comparison	TOST bound	p
MC	cel+randN vs real	0.0208	1
MC	celW+connShuf vs real	0.0208	1
MC	cel-sample vs real	0.0208	1
MC	conn-shuf-only vs real	0.0208	1
MC	er+randN vs real	0.0208	1
MC	global-sign-pres vs real	0.0208	2.233e-35
MC	local-sign vs real	0.0208	0.0002375
MC	local-sign+binary vs real	0.0208	0.001472
MC	local-sign+flat vs real	0.0208	1.385e-36
MC	local-sign+sample vs real	0.0208	1
MC	real vs shuffle	0.0208	1
MC	real vs ws-p0.1+randN	0.0208	1
IPC	cel+randN vs real	0.01818	1
IPC	celW+connShuf vs real	0.01818	1
IPC	cel-sample vs real	0.01818	1
IPC	conn-shuf-only vs real	0.01818	1
IPC	er+randN vs real	0.01818	1
IPC	global-sign-pres vs real	0.01818	3.488e-12
IPC	local-sign vs real	0.01818	0.01229
IPC	local-sign+binary vs real	0.01818	0.2554
IPC	local-sign+flat vs real	0.01818	6.347e-29
IPC	local-sign+sample vs real	0.01818	1
IPC	real vs shuffle	0.01818	1
IPC	real vs ws-p0.1+randN	0.01818	1
KR	cel+randN vs real	0.01771	1
KR	celW+connShuf vs real	0.01771	1
KR	cel-sample vs real	0.01771	1
KR	conn-shuf-only vs real	0.01771	1
KR	er+randN vs real	0.01771	1
KR	global-sign-pres vs real	0.01771	6.441e-45
KR	local-sign vs real	0.01771	1
KR	local-sign+binary vs real	0.01771	8.566e-45
KR	local-sign+flat vs real	0.01771	1
KR	local-sign+sample vs real	0.01771	1
KR	real vs shuffle	0.01771	1
KR	real vs ws-p0.1+randN	0.01771	1
GR	cel+randN vs real	0.02783	1
GR	celW+connShuf vs real	0.02783	1
GR	cel-sample vs real	0.02783	1
GR	conn-shuf-only vs real	0.02783	1
GR	er+randN vs real	0.02783	1
GR	global-sign-pres vs real	0.02783	7.378e-56
GR	local-sign vs real	0.02783	1
GR	local-sign+binary vs real	0.02783	1.202e-56
GR	local-sign+flat vs real	0.02783	0.0080

Chapter 6

AI Usage Disclosure

Artificial intelligence tools, specifically ChatGPT (OpenAI, GPT-5 series) and Codex (OpenAI), were used as assistive tools during the course of this work.

AI tools were used to review text for clarity, grammar, and structure, and to identify ambiguous or overstated claims. All text included in this manuscript was written, reviewed, and edited by the authors, and any AI-assisted edits were manually verified and revised as necessary to ensure accuracy.

AI tools were also used to assist in identifying relevant literature. These tools were used in a search-assistance capacity to locate papers related to reservoir computing and recurrent biological networks. All cited papers were independently located, read, and verified by the author prior to inclusion.

In addition, AI tools were used as a programming reference to identify relevant libraries, functions, and debugging strategies. They were also used to assist in generating example plotting code and suggesting implementation approaches. All code used in this project was written, reviewed, and validated by the author. Any external repositories referenced are explicitly cited.

AI tools were used solely as assistive instruments. All scientific claims, analysis, interpretations, and conclusions presented in this work are the independent work of the author.